

4.1 Кластеризація

У цьому розділі розглядаються методи кластеризації – алгоритми машинного навчання без учителя (Unsupervised). Тут модель немає готових правильних відповідей чи міток класів. Вона групує об'єкти (точки) разом виключно на основі їхньої схожості. Як міру схожості використовують відстань між точками (чим ближче точки одна до одної на графіку, тим вони схожіші).

4.1.1 К-середні (K-Means)

Алгоритм K-Means розбиває дані на декілька груп (кластерів, їх кількість можна задати самостійно) та автоматично шукає центри цих груп (центроїди).

Перед початком роботи необхідно підключити бібліотеки:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

- **make_blobs** – генератор штучних даних для задач кластеризації. Дозволяє створювати групи точок із заданою кількістю кластерів;
- **KMeans** – реалізація алгоритму K-Means, який автоматично шукає центри кластерів та розподіляє об'єкти між ними.

Математика відстаней та генерація даних

Перед застосуванням алгоритму K-Means необхідно підготувати дані для кластеризації.

```
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60,
random_state=42)

plt.figure(figsize=(6, 4))
plt.scatter(X[:, 0], X[:, 1], s=30, color='blue', alpha=0.7)
plt.title("Вхідні дані (до кластеризації)")
plt.xlabel("Ознака 1")
plt.ylabel("Ознака 2")
plt.show()
```

У прикладі використовується функція `make_blobs()` для генерації штучного набору даних: `n_samples` - визначає кількість точок у наборі даних; `centers` - кількість кластерів, навколо яких будуть генеруватися точки; `cluster_std` - рівень розкиду точок навколо центру кластера (велике значення робить групи менш щільними і може ускладнити кластеризацію).

Основою більшості алгоритмів кластеризації є обчислення відстаней між об'єктами. Алгоритм K-Means найчастіше використовує евклідову відстань.

Евклідова відстань показує, наскільки далеко одна точка знаходиться від іншої у просторі ознак. Для двох точок:

$$A(x_1, y_1), \quad B(x_2, y_2)$$

Відстань обчислюється за формулою:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Чим меншою є відстань - тим більш схожими вважаються об'єкти. Саме на основі цих відстаней K-Means визначає, до якого кластера належить кожна точка.

генеруються штучні дані для кластеризації, створюються 4 окремі групи точок, будується графік початкових даних без кластеризації.

Функція `scatter()`, тут: `X[:, 0]` - значення першої ознаки; `X[:, 1]` - значення другої ознаки; `s` - розмір точок; `alpha` - прозорість точок (допомагає краще бачити області, де точки накладаються одна на одну).

Аналіз сирих даних перед кластеризацією допомагає побачити структуру даних, оцінити кількість можливих кластерів, перевірити, наскільки добре групи відокремлені одна від одної.

Прив'язка точок до центроїдів (Expectation)

На етапі очікування (Expectation) алгоритм K-Means виконує розподіл кожної точки між кластерами на основі відстані до центроїдів. Спочатку центроїди ініціалізуються випадковим чином, після того починається ітераційний процес уточнення кластерів.

```
def compute_distances(X, centroids):  
    return np.sqrt(((X[:, np.newaxis, :] - centroids[np.newaxis, :, :]) **  
2).sum(axis=2)))
```

Функція `compute_distances()` обчислює евклідову відстань між кожною точкою та кожним центроїдом:

$$d = \sqrt{\sum_{i=1}^n (x_i - c_i)^2}$$

де:

- x_i - координати точки;
- c_i - координати центроїда.

Завдяки використанню `broadcasting` у NumPy усі відстані обчислюються одразу у вигляді матриці без використання циклів, що суттєво прискорює роботу алгоритму.

```
def kmeans_single_run(X, K, max_iter=300):
    N, D = X.shape
    random_indices = np.random.choice(N, K, replace=False)
    centroids = X[random_indices]

    for _ in range(max_iter):
        old_centroids = centroids.copy()

        distances = compute_distances(X, centroids)
        labels = np.argmin(distances, axis=1)

        for k in range(K):
            if np.any(labels == k):
                centroids[k] = X[labels == k].mean(axis=0)

        if np.allclose(old_centroids, centroids):
            break

    final_distances = compute_distances(X, centroids)
    min_distances = np.min(final_distances, axis=1)
    cost = np.sum(min_distances ** 2)

    return centroids, labels, cost
```

У функції `kmeans_single_run()` реалізовано повний цикл роботи K-Means: спочатку випадково обираються початкові центроїди, на кожній ітерації для кожної точки обчислюється відстань до всіх центроїдів, кожна точка прив'язується до найближчого центроїда (`labels`), після цього починається підготовка до етапу максимізації, де оновлюються центроїди.

Новий центроїд визначається формулою:

$$c_k = \frac{1}{N_k} \sum_{i=1}^{N_k} x_i$$

де:

N_k - кількість точок у кластері;

x_i - точки кластера;

c_k - новий центроїд.

Критично важливим моментом є використання `np.argmin(distances, axis=1)`, яке визначає найближчий кластер для кожної точки. Масив `distances` містить матрицю розміром (кількість точок) $\times$ (кількість центроїдів). Тут кожен рядок відповідає окремій точці, а кожен стовпець - певному центроїду.

Після завершення ітерації обчислюється загальна вартість кластеризації. Функція вартості показує, наскільки далеко точки знаходяться від своїх центроїдів. Алгоритм мінімізує значення, щоб зробити значення функції вартості якомога меншим:

$$J = \sum_{i=1}^N \|x_i - c_i\|^2$$

де:

x_i - точка;

c_i - відповідний центроїд.

Також виконується перевірка збіжності через `np.allclose(old_centroids, centroids)`, що дозволяє зупинити алгоритм, якщо центроїди більше не змінюються.

Перерахунок центроїдів та мінімізація вартості (Maximization)

На етапі максимізації (Maximization) алгоритм оновлює положення центроїдів на основі вже сформованих кластерів та обчислює якість поточного розбиття.

Функція вартості (`cost`) використовується для оцінки якості кластеризації: чим менше значення, тим компактніші кластери.

```
def fit_kmeans(X, K, n_init=50):  
    best_cost = float('inf')  
    best_centroids = None  
    best_labels = None  
  
    for _ in range(n_init):  
        centroids, labels, cost = kmeans_single_run(X, K)  
        if cost < best_cost:  
            best_cost = cost  
            best_centroids = centroids  
            best_labels = labels  
  
    return best_centroids, best_labels, best_cost  
  
K_clusters = 4  
custom_centroids, custom_labels, custom_cost = fit_kmeans(X, K=K_clusters,  
n_init=50)  
  
print(f"--- Результати кастомного K-means ---")  
print(f"Фінальна вартість моделі (Cost): {custom_cost:.2f}")
```

Функція `fit_kmeans()` реалізує багаторазовий запуск алгоритму K-Means з різними випадковими ініціалізаціями. Це необхідно, оскільки алгоритм чутливий до початкового вибору центроїдів і може сходитися до локального мінімуму.

Параметр `n_init=50` означає, що алгоритм запускається 50 разів, після чого обирається найкращий результат за значенням функції вартості (`cost`).

У фінальній частині задається кількість кластерів `K_clusters = 4`, після чого запускається повна процедура кластеризації. У результаті отримуються: координати фінальних центроїдів, мітки кластерів для кожної точки та значення функції вартості.

Проблема локальних оптимумів та фінальна візуалізація

Після реалізації власного K-Means алгоритму важливо порівняти його результат із реалізацією із бібліотеки `sklearn`, а також проаналізувати одну з ключових проблем методу - залежність від початкової ініціалізації центроїдів.

```
sklearn_kmeans = KMeans(n_clusters=K_clusters, n_init=50, random_state=42)  
sklearn_labels = sklearn_kmeans.fit_transform(X)  
sklearn_preds = sklearn_kmeans.labels_  
sklearn_centroids = sklearn_kmeans.cluster_centers_
```

У цьому фрагменті використовується реалізація KMeans із бібліотеки sklearn, тут: `n_clusters` – задає кількість кластерів, `n_init` – алгоритм запускається 50 разів із різними початковими центроїдами.

Метод `fit_transform(X)` виконує навчання моделі та обчислення відстаней до кластерів, а `labels_` містить фінальні мітки кластерів для кожної точки. Центроїди після навчання зберігаються у `cluster_centers_`.

K-Means є алгоритмом, який мінімізує функцію вартості ітеративно, тому він не гарантує знаходження глобального мінімуму. Через випадкову ініціалізацію центроїдів алгоритм може: застрягти в локальному мінімумі, отримати неоптимальний розподіл кластерів, дати різні результати при різних запусках. Це одна з ключових слабких сторін K-Means.

Щоб зменшити вплив випадкової ініціалізації, використовується підхід багаторазового запуску алгоритму (Multi-initialization): K-Means запускається кілька разів (50-100), кожного разу з різними початковими центроїдами, обирається результат з найменшою функцією вартості (cost). Саме це реалізовано через параметр `n_init`, який підвищує стабільність та якість фінального результату.

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=custom_labels, s=30, cmap='viridis',
            alpha=0.7)
plt.scatter(custom_centroids[:, 0], custom_centroids[:, 1], c='red', s=150,
            marker='X', label='Центроїди')
plt.title(f"Кастомний K-means (Cost: {custom_cost:.1f})")
plt.xlabel("Ознака 1")
plt.ylabel("Ознака 2")
plt.legend()

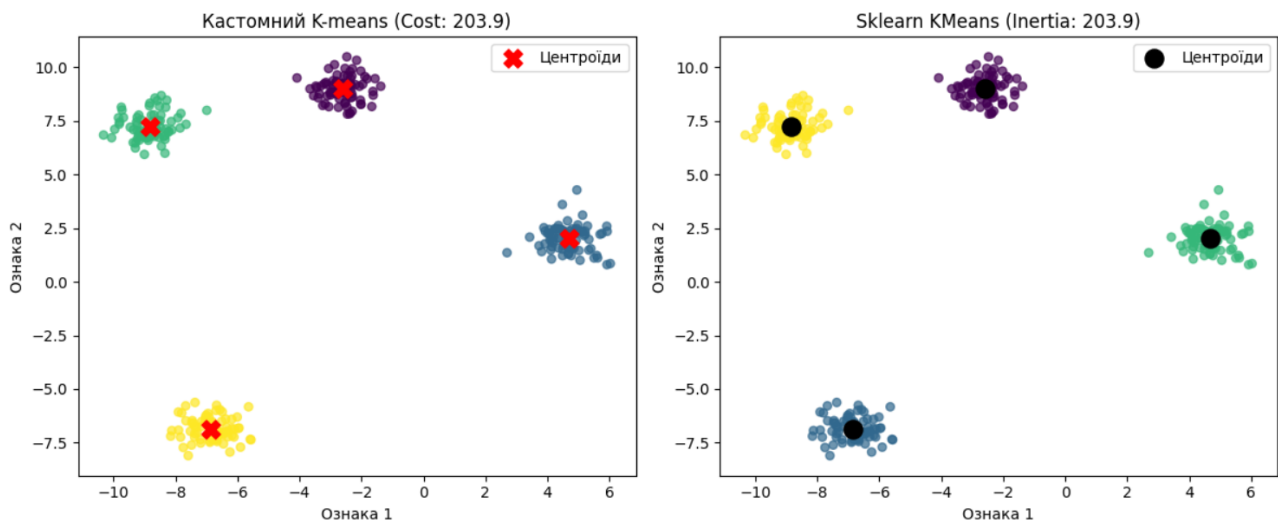
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=sklearn_preds, s=30, cmap='viridis',
            alpha=0.7)
plt.scatter(sklearn_centroids[:, 0], sklearn_centroids[:, 1], c='black',
            s=150, marker='o', label='Центроїди')
plt.title(f"Sklearn KMeans (Inertia: {sklearn_kmeans.inertia_:.1f})")
plt.xlabel("Ознака 1")
plt.ylabel("Ознака 2")
plt.legend()

plt.tight_layout()
plt.show()
```

У цьому фрагменті буде порівняльна візуалізація двох підходів: зліва показано результат кастомної реалізації K-Means, справа – результат реалізації з бібліотеки sklearn.

Кожна точка на графіку розфарбовується відповідно до свого кластера, а центроїди виділяються окремими маркерами: X (червоний маркер) - центроїди кастомної реалізації, круги (чорний маркер) - центроїди sklearn-моделі.

Для оцінки якості кластеризації використовуються: `cost` - у кастомній кластеризації, `inertia_` - у sklearn. Обидві метрики розраховують функцію вартості.



Таким чином стає видно, що навіть проста реалізація K-Means дає результат, близький до sklearn, за умови правильної ініціалізації та багаторазового запуску. Насправді такий підхід більше використовується при навчанні - щоб зрозуміти логіку. Для проектів частіше беруть готовий алгоритм з бібліотеки.

4.1.2 Ієрархічна кластеризація (Hierarchical Clustering)

Ієрархічна кластеризація працює за принципом «знизу-вгору»: кожна точка спочатку є окремим кластером, далі найближчі кластери поступово об'єднуються. Процес триває до отримання великого кластера або заданої кількості груп. Результат можна представити у вигляді дерева об'єднань - дендрограми, яка показує структуру даних на різних рівнях.

Перед початком роботи необхідно підключити бібліотеки:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
```

- **AgglomerativeClustering** - реалізація агломеративної (знизу-вгору) кластеризації зі sklearn;
- **dendrogram і linkage** - інструменти для побудови дендрограми, яка відображає структуру об'єднання кластерів.

Цей рядок налаштовує стиль візуалізації графіків:

```
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in
plt.style.available else 'default')
```

це дозволяє зробити графіки більш читабельними.

Принцип об'єднання та типи зв'язків (Linkage)

Алгоритм багаторазово об'єднує найближчі кластери відповідно до вибраного критерію зв'язку (linkage).

```
X_blobs, y_blobs = make_blobs(n_samples=150, n_features=2, centers=5,
cluster_std=0.60, random_state=42)

Z = linkage(X_blobs, method='ward')
```

Після генерації штучного набору даних (make_blobs) використовується функція linkage(), яка виконує побудову ієрархічної структури кластерів. Вона обчислює відстані між кластерами, визначає порядок їх об'єднання та формує матрицю зв'язків для побудови дендрограми.

Критерій зв'язку визначає, як саме алгоритм обчислює відстань між двома кластерами. Метод **ward** мінімізує приріст внутрішньокластерної дисперсії, намагається створювати компактні та збалансовані кластери, найкраще працює для сферичних груп даних¹.

Математично метод прагне мінімізувати:

$$W = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

¹ Сферичні групи даних - використовують для обробки даних з лазерних сканерів або радарів.

де:

- C_k – кластер;
- x_i – точки кластера;
- μ_k – центр кластера.

Чим менше значення – тим компактніше є кластер.

Метод **complete** визначає відстань між кластерами як відстань між двома найвіддаленішими точками кластерів. Формула:

$$D(A, B) = \max_{a \in A, b \in B} d(a, b)$$

Такий підхід створює компактні кластери але може бути чутливим до викидів (поодинокі точки даних, сильно віддалені від основної маси об'єктів).

Метод **average** використовує середню відстань між усіма парами точок двох кластерів. Формула:

$$D(A, B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a, b)$$

де: $|A|$ та $|B|$ – кількість точок у кластерах.

Цей метод менш чутливий до шуму та створює більш збалансовані групи.

Вибір критерію linkage суттєво впливає на результат кластеризації: ward – створює компактні, приблизно однакові за формою кластери, complete – щільні та добре відокремлені групи, average – більш збалансоване й гнучке об'єднання кластерів.

Візуалізація дерева та фіксація кластерів

Дендрограма – це дерево, яке показує, як алгоритм поступово об'єднує об'єкти в кластери. Кожне об'єднання двох кластерів малюється лінією, висота цієї лінії показує, наскільки кластери були далекими один від одного (малюнок нижче).

```
plt.figure(figsize=(10, 5))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=45,
leaf_font_size=10)
plt.title("Дендрограма ієрархічної кластеризації (Ward's Method)")
plt.xlabel("Індекси точок або розміри піддерев")
plt.ylabel("Відстань об'єднання (Distance)")
plt.axhline(y=7.5, color='r', linestyle='--', label='Лінія розрізу на 5
кластерів')
plt.legend()
plt.show()
```

Функція `dendrogram()` візуалізує дерево об'єднань кластерів, тут: по осі X розташовані точки або піддерева, по осі Y показано відстань, на якій відбулося об'єднання кластерів. Параметри функції: `truncate_mode` - скорочує дендрограму та показує лише останні об'єднання кластерів; `p` - кількість останніх вузлів, які будуть показані; `leaf_rotation` - поворот підписів листків дерева; `leaf_font_size` - розмір шрифту.



Тобто ось дендрограма - об'єднання починається знизу, від самих менших квадратиків до більших-більших - на етапі коли залишилось 5 кластерів ми сказали програмі спинитись, так би вона утворила один великий кластер.

У цьому фрагменті створюється фінальна модель кластеризації:

```
agg_clustering = AgglomerativeClustering(n_clusters=5, linkage='ward')
agg_labels = agg_clustering.fit_predict(X_blobs)
```

Тут: `n_clusters` - кількість кластерів, `linkage` - критерій зв'язку. Метод `fit_predict()` одночасно навчає модель та призначає мітки кожній точці.

```
plt.figure(figsize=(6, 4))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], c=agg_labels, cmap='tab10', s=40,
            alpha=0.8, edgecolor='k')
plt.title("Результат Agglomerative Clustering (K=5)")
plt.xlabel("Ознака 1")
plt.ylabel("Ознака 2")
plt.show()
```

У фінальному графіку: `cmap` - задає кольорову палітру для кластерів; `edgecolor` - додає чорні контури навколо точок.

Дендрограма дозволяє дослідити структуру даних, оцінити схожість кластерів, визначити оптимальну кількість груп та зрозуміти процес об'єднання точок у кластери.

4.1.3 Щільнісна кластеризація (DBSCAN)

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) - це алгоритм, який формує кластери на основі щільності розташування точок у просторі. Переваги: автоматично знаходить кількість кластерів, виділяє шумові точки, працює з нелінійними структурами даних.

Перед початком роботи необхідно підключити бібліотеки:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
```

- `make_moons` - генератор нелінійних даних у формі двох «півмісяців»;
- `DBSCAN` - реалізація алгоритму щільнісної кластеризації.

Анатомія точок та параметри щільності

На цьому етапі формується набір даних і задаються ключові параметри DBSCAN, які визначають, як саме алгоритм буде знаходити щільні області та відокремлювати шум.

```
X_moons, y_moons = make_moons(n_samples=250, noise=0.07, random_state=42)

dbscan = DBSCAN(eps=0.2, min_samples=5)
db_labels = dbscan.fit_predict(X_moons)
```

Функція `make_moons()` генерує нелінійний набір даних у вигляді двох «півмісяців». Тут: `noise` - додає невеликий шум, що робить дані менш ідеальними та більш наближеними до реальних.

Далі створюється модель DBSCAN, тут: `eps` - визначає максимальну відстань між точками, щоб вони вважалися сусідами (значення занадто мале - багато шуму, занадто велике - кластери зливаються); `min_samples` - задає мінімальну кількість точок у межах `eps`, щоб область вважалась щільною (включає саму точку та її сусідів, якщо умова виконується - точка може бути ядром кластера).

DBSCAN розділяє всі точки на три типи залежно від щільності їхнього оточення: `ядрові точки (Core points)` - навколо неї знаходиться достатня кількість сусідів у заданому радіусі; `граничні точки (Border points)` - знаходяться поруч із ядровими але самі не мають достатньої кількості сусідів; `шум (Noise points)` - не належать жодному кластеру (такі точки DBSCAN позначає `label=-1`).

Складна геометрія та фільтрація аномалій

На цьому етапі виконується аналіз DBSCAN, виділення кластерів а також візуальне розділення `core-`, `border-` і `noise-`точок.

```
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True
```

У цьому фрагменті створюється маска, яка визначає які точки є ядровими. `dbscan.core_sample_indices_` містить індекси точок, які мають достатню щільність у своєму колі. Таким чином `True` - ядрова точка, `False` - гранична точка або шум.

```
n_clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise = list(db_labels).count(-1)
```

У цьому блоці: `n_clusters` - кількість знайдених кластерів (шум не враховується), `n_noise` - кількість точок, які DBSCAN класифікував як аномалії.

Після виконання кластеризації відбувається підрахунок кількості знайдених кластерів та шумових точок:

```
print(f"Знайдено стійких кластерів: {n_clusters}")
print(f"Кількість виявлених точок шуму (аномалій): {n_noise}")
```

Далі виконується візуалізація результату кластеризації, де кожен кластер відображається окремим кольором, а шумові точки виділяються окремо:

```
plt.figure(figsize=(8, 5))
unique_labels = set(db_labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1,
len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
        col = [0, 0, 0, 1]

    class_member_mask = (db_labels == k)

    xy_core = X_moons[class_member_mask & core_samples_mask]
    plt.scatter(xy_core[:, 0], xy_core[:, 1], facecolors=tuple(col),
                edgecolor='k', s=60, label=f'Кластер {k}' if k != -1 else
'Шум')

    xy_border = X_moons[class_member_mask & ~core_samples_mask]
    plt.scatter(xy_border[:, 0], xy_border[:, 1], facecolors=tuple(col),
                edgecolor='k', s=20, alpha=0.6)

plt.title(f"DBSCAN Кластеризація (Кластерів: {n_clusters}, Точок шуму:
{n_noise})")
plt.xlabel("Ознака 1")
plt.ylabel("Ознака 2")
plt.legend(loc='upper right')
plt.show()
```